Compiling λ-Calculus to SECD-Machine Instructions

Diogo Gomes de Araújo (202510267)

March 25, 2026

Contents

1 Introduction **1**

2 SECD-Machine **1**

3 Parsing **2**
3.1 Example 2

4 Compilation **2**
4.1 Source Language 2
4.2 Target Language 3
4.3 Example 3

5 Evaluation **4**
5.1 Example 5

6 Conclusion **5**

1 Introduction

This report was produced as an assignment for the Implementation of Programming Languages course at Faculdade de Ciências da Universidade do Porto. It documents the design and implementation of a compiler from an extended λ-Calculus [1] inspired language to a stack-based abstract machine, written in OCaml. The source code is publicly available in the following repository.

2 SECD-Machine

In 1964, Peter Landin specified a foundational model for the computation of simple arithmetic expressions [2] using Church's λ-Calculus as a chain of transitions over a state until a final value is reached.

The state was a tuple containing: a stack `s` of intermediate *values* produced during evaluation; an environment `e` mapping variable names to their respective values; a control list `c` of instructions and expressions awaiting evaluation; and a dump `d` that saves the current machine state (stack, environment, and control) at each function call, restoring it upon return.

Since Landin's publication, many abstract machines have been proposed with different state compositions. This project extends the original machine with an additional memory store `m`, which holds *closures* (pairings of a λ-term with the environment active at the time of its creation) allowing environments to map variable names to memory addresses rather than values directly. The resulting state tuple is:

$$(\mathtt{s} * \mathtt{e} * \mathtt{c} * \mathtt{d} * \mathtt{m}) \rightarrow \mathtt{value} \tag{1}$$

The machine halts when the control list `c` and dump `d` are both empty, at which point the stack `s` holds a single final value, corresponding to the result of the computation.

Additionally, SECD evaluates instructions using a *call-by-value* strategy, where arguments to a function are fully evaluated before the function itself.

3 Parsing

To facilitate generation and improve the readibility of λ-terms, a simple parser was created using Menhir [5], an LR(1) parser generator for OCaml.

3.1 Example

The term demonstrated assigns a λ-function `incr` that increments an argument `x` by 1 followed by the application of 1 to `incr`:

```
    let incr =
        \x -> x + 1
    in incr 1
```

4 Compilation

To execute programs on the SECD-machine, we must translate a language into a sequence of instructions that it can understand and execute. This requires defining both the source and target languages, along with a set of rules that map one to the other.

4.1 Source Language

The source language used in this project is an extended version of λ-Calculus, defined through the following types:

```
type identity = string
type term =
| Variable of identity
| Constant of int
| Lambda of identity * term
| Application of term * term
| IfZero of term * term * term
| Let of identity * term * term
| Addition of term * term
| Subtraction of term * term
| Multiplication of term * term
| Division of term * term
| Fix of term
```

A `term` is either a `Variable`, referring to a named binding; a `Constant` integer value; a `Lambda` abstraction pairing a bound variable with a body term; or an `Application` of one term to another. The language is further extended with `IfZero`, a conditional that branches on whether a term evaluates to zero; `Let`, which binds the result of a term to a name within a scope; the standard arithmetic operations `Addition`, `Subtraction`, `Multiplication`, and `Division`; and finally `Fix`, a fixed-point combinator that enables the definition of recursive functions.

4.2 Target Language

The target language of the compiler is a sequence of stack-machine instructions based on Henderson's virtual machine for compiling Lisp/Scheme [4], defined as follows:

```
type instr =
| HALT
| LDC of int
| LD of int
| ADD
| SUB
| MUL
| DIV
| SEL of instr list * instr list
| JOIN
| LDF of instr list
| LDFR of instr list
| AP
| AA
| LDRF
| RTN
```

An `instr` is one of the following: `HALT`, which terminates the machine; `LDC`, which loads a constant integer onto the stack; and `LD`, which loads a value from the current environment by index. The arithmetic operations `ADD`, `SUB`, `MUL`, and `DIV` each pop two values from the stack and push their result. `SEL` implements conditional branching by selecting between two instruction sequences depending on whether the top of the stack is zero, while `JOIN` returns control to the instruction sequence saved in the dump after a branch. `LDF` loads a λ-abstraction as a closure onto the stack, and `LDFR` does the same for recursive functions. `AP` applies a closure to the current argument, saving the current state to the dump, while `AA` moves a value from the temporary stack to the environment. `LDRF` loads a recursive function reference from the environment, and `RTN` returns from a function call by restoring the previously saved state from the dump.

4.3 Example

Consider the term bellow, generated by parsing the expression shown in the previous example:

```
Let ("incr", Lambda ("x", Addition (Variable "x", Constant 1)),
Application (Variable "incr", Constant 1))
```

The λ-abstraction is compiled into a closure using `LDF`, whose body evaluates the parameter and the constant, performs the addition, and returns:

$$\texttt{LDF [LD 0; LDC 1; ADD; RTN]} \tag{2}$$

The `Let` binding extends the environment by associating the computed closure with `incr` through `AA`. The application is then compiled by first loading the function from the environment, followed by the argument, and finally invoking the application instruction:

$$\texttt{LD 0; LDC 1; AP} \tag{3}$$

Thus, the complete compiled code is:

$$\texttt{[LDF [LD 0; LDC 1; ADD; RTN]; AA; LD 0; LDC 1; AP]} \tag{4}$$

5 Evaluation

In order to extract a final value from a given λ-term, an internal transition function from state to state, called `execute`, is applied recursively in `execute_t`, until the control list (`c`) is empty and the stack has only an element left, that represents the result of the computation. The evaluation functions' signatures are:

```
val execute: conf -> conf
val execute_t: conf -> value
```

At each iteration, the SECD-state is evaluated against a common set of rules, defined below, to determine the state of the subsequent recursive step:

$$\frac{}{(s, e, \text{LD } i :: c, d, m) \to ((e_i :: s), e, c, d, m)}$$

$$\frac{}{(s, e, \text{LDC } n :: c, d, m) \to ((\text{Int } n :: s), e, c, d, m)}$$

$$\frac{}{((\text{Int } v_1 :: \text{Int } v_2 :: s), e, \text{ADD} :: c, d, m) \to ((\text{Int } (v_2 + v_1) :: s), e, c, d, m)}$$

$$\frac{}{((\text{Int } v_1 :: \text{Int } v_2 :: s), e, \text{SUB} :: c, d, m) \to ((\text{Int } (v_2 - v_1) :: s), e, c, d, m)}$$

$$\frac{}{((\text{Int } v_1 :: \text{Int } v_2 :: s), e, \text{MUL} :: c, d, m) \to ((\text{Int } (v_2 \cdot v_1) :: s), e, c, d, m)}$$

$$\frac{}{((\text{Int } v_1 :: \text{Int } v_2 :: s), e, \text{DIV} :: c, d, m) \to ((\text{Int } (v_2 / v_1) :: s), e, c, d, m)}$$

$$\frac{a = \text{next}(m)}{(s, e, \text{LDF } c' :: c, d, m) \to ((\text{Address } a :: s), e, c, d, m[a \mapsto (c', e)])}$$

$$\frac{}{(v :: s, e, \text{AA} :: c, d, m) \to (s, v :: e, c, d, m)}$$

$$\frac{m(a) = (c', e')}{(v :: \text{Address } a :: s, e, \text{AP} :: c, d, m) \to ([], v :: e', c', (s, e, c) :: d, m)}$$

$$\frac{}{(v :: s, e, \text{RTN} :: c, (s', e', c') :: d, m) \to (v :: s', e', c', d, m)}$$

$$\frac{}{((\text{Int } 0 :: s), e, \text{SEL } (c_1, c_2) :: c, d, m) \to (s, e, c_1, ([], [], c) :: d, m)}$$

$$\frac{}{((\text{Int } n :: s), e, \text{SEL } (c_1, c_2) :: c, d, m), n \neq 0 \to (s, e, c_2, ([], [], c) :: d, m)}$$

$$\frac{}{(s, e, \text{JOIN} :: c, (_, _, c') :: d, m) \to (s, e, c', d, m)}$$

$$\frac{a = \text{next}(m)}{(s, e, \text{LDFR } c' :: c, d, m) \to ((\text{Address } a :: s), e, c, d, m[a \mapsto (c', (\text{Address } a :: e))])}$$

5.1 Example

Consider the compiled SECD-instructions from the previous example. The initial configuration is:

$$(s_0, e_0, c_0, d_0, m_0) = ([], [], [\texttt{LDF [LD 0; LDC 1; ADD; RTN]; AA; LD 0; LDC 1; AP}], [], m)$$

Execution begins with the `LDF` instruction, which creates a closure from the code sequence `[LD 0; LDC 1; ADD; RTN]`, stores it in memory at address a, and pushes that address onto the stack:

$$\frac{}{([], e_0, \texttt{LDF [LD 0; LDC 1; ADD; RTN]} :: c_1, d_0, m_0) \to ([\text{Address } a], e_0, c_1, d_0, m_1[a \mapsto ([\texttt{LD 0; LDC 1; ADD; RTN}], e_0)])}$$

Next, the `AA` instruction pops this address from the stack and installs it as the new environment, preparing the machine to access the closure:

$$\frac{}{([\text{Address } a], e_0, \texttt{AA} :: c_1, d_0, m_1) \to ([], [\text{Address } a], c_1, d_0, m_1)}$$

`LD 0` loads the first element of the current environment onto the stack, retrieving the closure address stored, so it is ready to be applied:

$$\frac{}{([], [\text{Address } a], \texttt{LD 0} :: c_1, d_0, m_1) \to ([\text{Address } a], [\text{Address } a], c_1, d_0, m_1)}$$

Argument 1 is pushed onto the stack using `LDC 1`. The `AP` instruction applies the closure (clears the stack for the new environment, sets up the closure's code and environment with the argument, and saves the current continuation onto the dump):

$$\frac{}{([\text{Address } a, \text{Int } 1], e, \texttt{AP} :: c_1, d_0, m_1) \to ([], [\text{Int } 1], [\texttt{LD 0; LDC 1; ADD; RTN}], ([\,], e, c_1) :: d_0, m_1)}$$

Inside the closure's body, `LDC 1` pushes the constant 1 onto the stack, so the stack now contains both the argument and the constant:

$$\frac{}{([\text{Int } 1], [\text{Int } 1], \texttt{LDC 1} :: c', d, m) \to ([\text{Int } 1, \text{Int } 1], e, c', d, m)}$$

The `ADD` instruction then pops these two values, computes their sum $1 + 1 = 2$, and pushes the result back onto the stack:

$$\frac{}{([\text{Int } 1, \text{Int } 1], e, \texttt{ADD} :: c', d, m) \to ([\text{Int } 2], e, c', d, m)}$$

Finally, `RTN` returns from the closure by restoring the saved continuation and environment from the dump and delivering the computed value 2 to the caller:

$$\frac{}{([\text{Int } 2], e, \texttt{RTN} :: c', (s', e', c'') :: d, m) \to ([\text{Int } 2] :: s', e', c'', d, m)}$$

The computation halts with a final result on the stack:

$$[\text{Int } 2] \Rightarrow 2$$

6 Conclusion

This project implements a parser for a minimal functional programming language inspired by λ-Calculus, along with a compiler that translates programs into SECD machine instructions and a virtual machine that executes them. Through this work, I developed a solid understanding of how call-by-value functional languages are implemented, from parsing and compilation to runtime evaluation.

References

[1] Alonzo Church. *A set of postulates for the foundation of logic.* Annals of Mathematics, 33(2):346–366, 1932.

[2] Peter J. Landin. The mechanical evaluation of expressions. *The Computer Journal*, 6(4):308–320, 1964.

[3] Olivier Danvy. A rational deconstruction of Landin's SECD machine. In *Proceedings of the Symposium on Implementation and Application of Functional Languages*, pages 52–71. Springer, 2004.

[4] Peter Henderson. *Functional Programming: Application and Implementation.* Prentice Hall, 1980.

[5] François Régis-Gianas and Yann Pottier. *Menhir Reference Manual.* 2016.